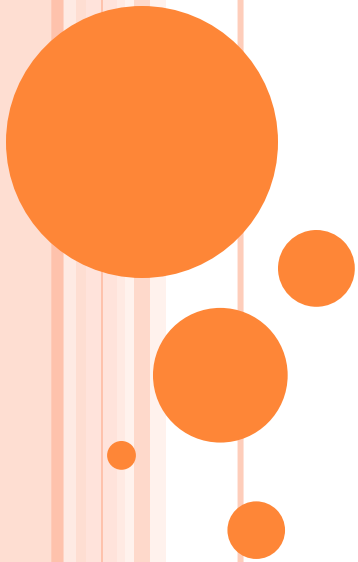


Chapter 5

STATE MODELING



STATE MODELING

- State model describes the sequences of operations that occur in response to external stimuli.
- The state model consists of multiple state diagrams, one for each class with temporal behaviour that is important to an application.
- The state diagram is a standard computer science concept that relates events and states.
- Events represent external stimuli and states represent values objects.



EVENTS

- ❑ An event is an occurrence at a point in time, such as user depresses left button.
 - ❑ Deccan flight departs from Bombay.
 - ❑ An event happens instantaneously with regard to time scale of an application.
 - ❑ One event may logically precede or follow another, or the two events may be unrelated (concurrent; they have no effect on each other).
 - ❑ Events include error conditions as well as normal conditions.
- 

- Example include *motor jammed*, *transaction aborted*, and *timeout* are typical error events.
- The term *event* is often used ambiguously. Sometimes it refers to an instance, at other times to a class.
- Three types of events:
 - signal event
 - change event
 - time event



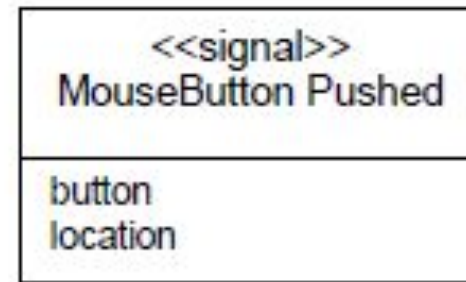
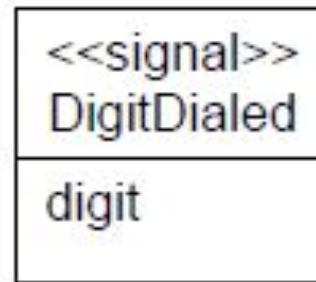
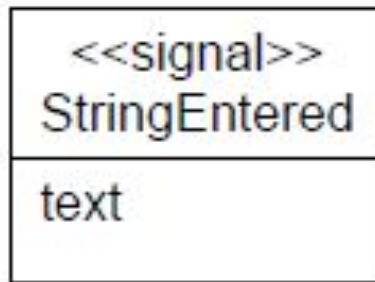
□ Signal Event

- A signal is an explicit one-way transmission of information from one object to another.
- It is different from a subroutine call that returns a value.
- An object sending a signal to another object may expect a reply, but the reply is a separate signal under the control of the second object, which may or may not choose to send it.
- A signal event is the event of sending or receiving a signal (concern about receipt of a signal).



- We are more concerned about the receipt of a signal, because it causes effects in the receiving

- Eg:

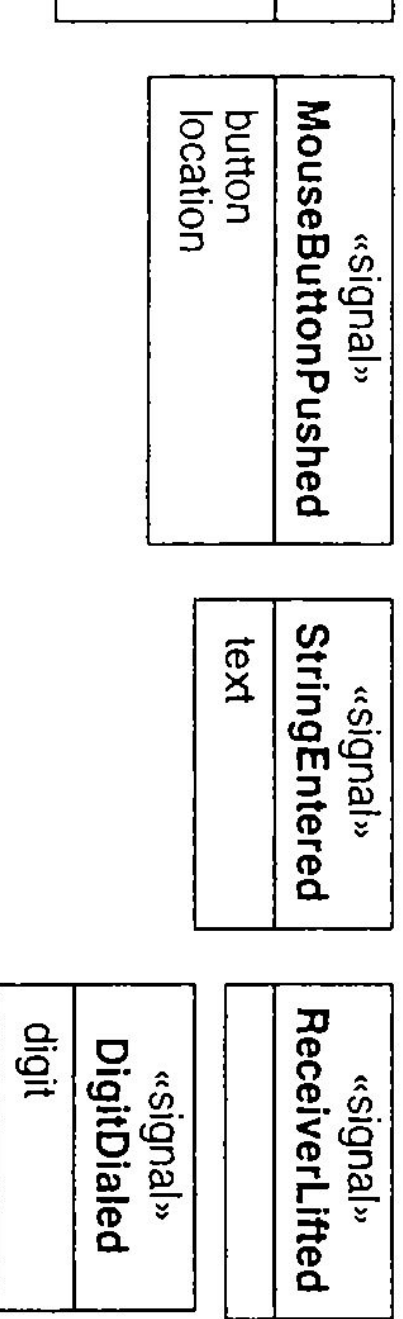


- **The difference between signal and signal event**
 - a signal is a message between objects
 - a signal event is an occurrence in time.





rence, but we
ch signal class
avior.



Signal classes and attributes. A signal is an explicit one-way transmission of information from one object to another.

- Every signal trar
group them into
name to indicate

- Ex: *UA flight 123 departs from Chicago on January 10, 1991* is an instance of signal class *Flight Departure*.
- Some signals are simple occurrences, but most signal classes have attributes indicating the values they convey.
- For Ex: *FlightDeparture* has attributes *airline, flightNumber, city, date*.
- The UML notation is the keyword *signal* in guillemets(<< >>) above the signal class name in the top section of a box.
- The second section lists the signal attributes.



❑ Change Event

- ❑ A change event is an event that is caused by the satisfaction of a Boolean expression.
- ❑ The intent of a continually tested false to true, the event
- ❑ UML notation for by a parenthesized

when (room temperature < heating set point)
when (room temperature > cooling set point)
when (battery power < lower limit)
when (tire pressure < minimum pressure)

vents. A change event is an event that is caused by the
n of a boolean expression.

t the expression is
ssion changes from

word when followed



❑ Time Event

- ❑ Time event is an event caused by the occurrence of an absolute time or the elapse of a time interval.
- ❑ UML notation for : followed by a parent
- ❑ The notation for : followed by a paren time duration.

■ when (date = January 1, 2000)
■ after (10 seconds)

nts. A time event is an event caused by the occurrence of
e time or the elapse of a time interval.

s the keyword when
involving time.

s the keyword after
n that evaluates to a



❑ States

- ❑ A state is an abstraction of the values and links of an object.
- ❑ Sets of values and links that define a state according to the given object.
- ❑ UML notation for state:
 - an optional state name in boldface, center the name and capitalize the first letter
 - BelowFreezing
 - Powered
 - Dialing
 - Waiting

A state is an abstraction of the values and links of an object.

combined together into a single behaviour of objects.

State box Containing the state name in the top of the box,



- In defining states, Ignore attributes that do not affect the behaviour of the object, and lump together in a single state all combinations of values and links with the same response of events.
- The objects in a class have a finite number of possible states-one or possibly some large number. Each object can only be in one state at a time.
- A state specifies the response of an object to input events. All events are ignored in a state, except those for which behavior is explicitly prescribed.
- The response may include the invocation of behavior or a change of state.



- For Ex: If a digit is dialled in state *DialTone*, the phone line drops the dial tone and enters state *Dialing*; if receiver is replaced in state *DialTone*, the phone line goes dead and enters state *idle*.
- All events are ignored in a state, except those for which behavior is explicitly prescribed.
- There is a certain symmetry between events and states.
 - Event represents points in time.
 - State represents intervals of time.
- A state corresponds to the interval between two events received by an object.



- For Ex: after the receiver is lifted and before the first digit is dialed, the phone line is in state DialTone.
- The state of an object depends on past events, which in most cases are eventually hidden by subsequent events.
- For Ex: events that happened before the phone is hung up do not affect future behavior; the idle state “forgets” events received prior to the receipt of the hang up signal.

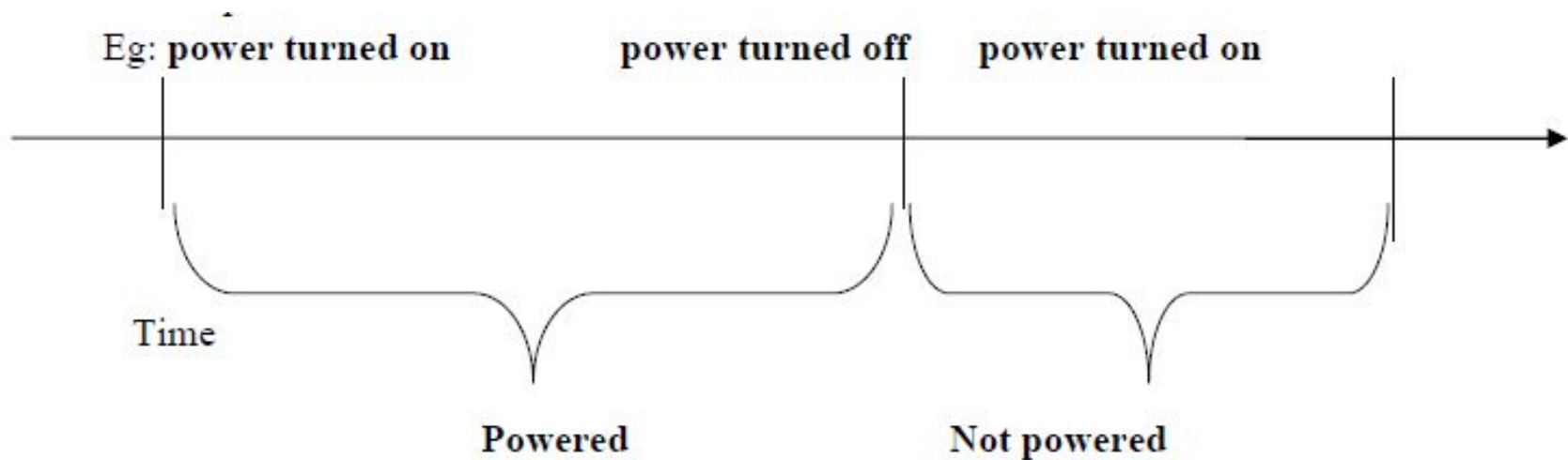


Fig: Event vs. state



- Both events and states depend on the level of abstraction.
- For Ex: a travel agent planning an itinerary would treat each segment of a journey as a single event;
- A flight status board in an airport would distinguish departures and arrivals; an air traffic control system would break each flight into many geographical legs.
- Ex below shows state Alarm ringing on a watch. The state has a suggestive name and a natural-language description of its purpose.



State Alarm ringing on a watch

- **State** : *Alarm Ringing*
- **Description** : alarm on watch is ringing to indicate target time
- **Event sequence that produces the state** :
 - setAlarm (targetTime)*
 - any sequence not including *clearAlarm*
 - when (*currentTime = targetTime*)
- **Condition that characterizes the state:**
 - alarm = on, alarm set to *targetTime*,
 - targetTime ≤ currentTime ≤ targetTime + 20 sec* , and no button has been pushed since *targetTime*
- **Events accepted in the state:**

event	response	next state
when (<i>currentTime = targetTime + 20</i>)	<i>resetAlarm</i>	<i>normal</i>
<i>buttonPushed</i> (any button)	<i>resetAlarm</i>	<i>normal</i>

- The event sequence that leads to the state consists of setting the alarm, doing anything that does not clear the alarm, and then having the target time occur.
- A declarative condition for the state is given in terms of parameters such as *current* and *target time*, the alarm stops ringing after 20 seconds.
- Finally, a stimulus-response table shows the effect of events *current Time* and *button pushed*, including the response that occurs and the next state.
- The different descriptions of the state may overlap.



TRANSITIONS & CONDITIONS

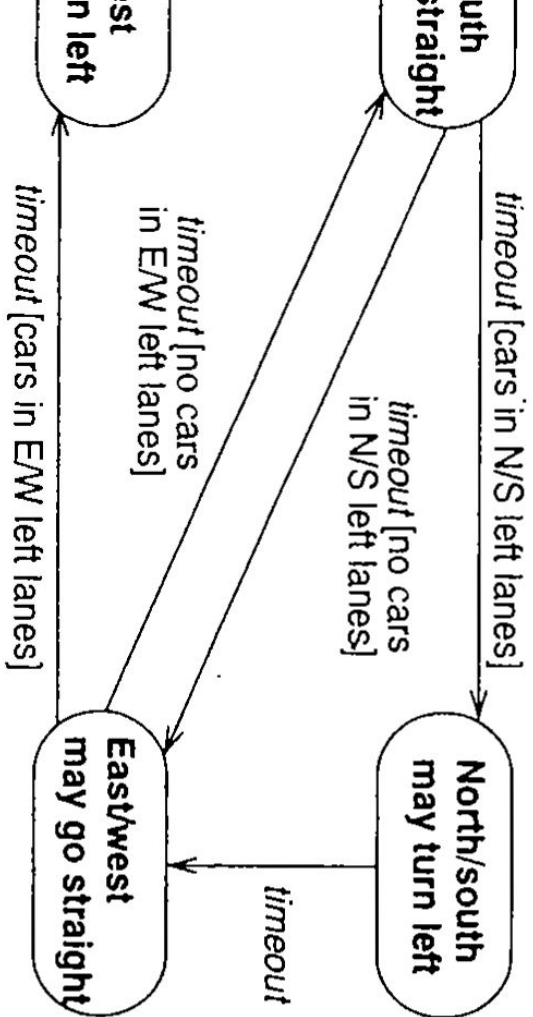
- A transition is an instantaneous change from one state to another.
- The transition is said to fire upon the change from the source state to target state.
- The origin and target of a transition usually are different states, but sometimes may be the same.
- A transition fires when its events (multiple objects) occurs.
- The choice of next state depends on both the original state and the event received.



- A guard condition is a Boolean expression that must be true in order for a transition to occur.
- A guard condition is checked only once, at the time the event occurs, and the transition fires if the condition is true.

Guard condition Vs. change event

Guard condition	change event
a guard condition is checked only once	a change event is checked continuously
UML notation for a transition is a line	may include event label in italics
followed by guard condition in square brackets	from the origin state to the target state an arrowhead points to the target state.



traffic lights at an intersection checks the east-west

Fig shows intersection. turn lanes.

If no car is in the north-south and/or east-west turn lanes, then the traffic light control logic is smart enough to skip the left turn portion of the cycle.

- The UML notation for a transition is a line from the origin state to the target state. An arrowhead points to the target state. The line may consist of several line segments.
- An event may label the transition and be followed by an optional guard condition in square brackets.
- By convention, we usually confine line segments to a rectilinear grid.
- We italicize the event name and show the condition in normal font.



STATE DIAGRAM

- A **state diagram** is a graph whose nodes are states and whose directed arcs are transitions between states.
- A state diagram specifies the state sequence caused by event sequences.
- State names must be unique within the scope of a state diagram.
- All objects in a class execute the state diagram for that class, which models their common behaviour.
- A **state model** consists of multiple state diagrams one state diagram for each class with important temporal behavior.



- ❑ State diagrams must match on their interfaces-events and guard conditions
- ❑ The individual state diagram interact by passing events and through the side effects of guard conditions.
- ❑ Some events and guard conditions appear in a single state diagram; other appear in multiple state diagrams.
- ❑ A class with more than one state has important temporal behavior .
- ❑ State diagrams interact by passing events and through the side effects of guard conditions.



Sample state Diagram

- UML notation for a state diagram is a rectangle with its name in small pentagonal tag in the upper left corner.
- The constituent states and transitions lie within the rectangle.
- States do not totally define all values of an object.
- For Ex: state *Dialing* includes all sequences of incomplete phone numbers.
- It is not necessary to distinguish between different numbers as separate states, since they all have the same behavior, but the actual number dialed must be saved as an attribute.



- If more than one transition leaves a state, then the first event to occur causes the corresponding transition to fire.
- If an event occurs and no transition matches it, then the event is ignored.
- If more than one transition matches an event, only one transition will fire, but the choice is nondeterministic.



Eg: Sample state diagram

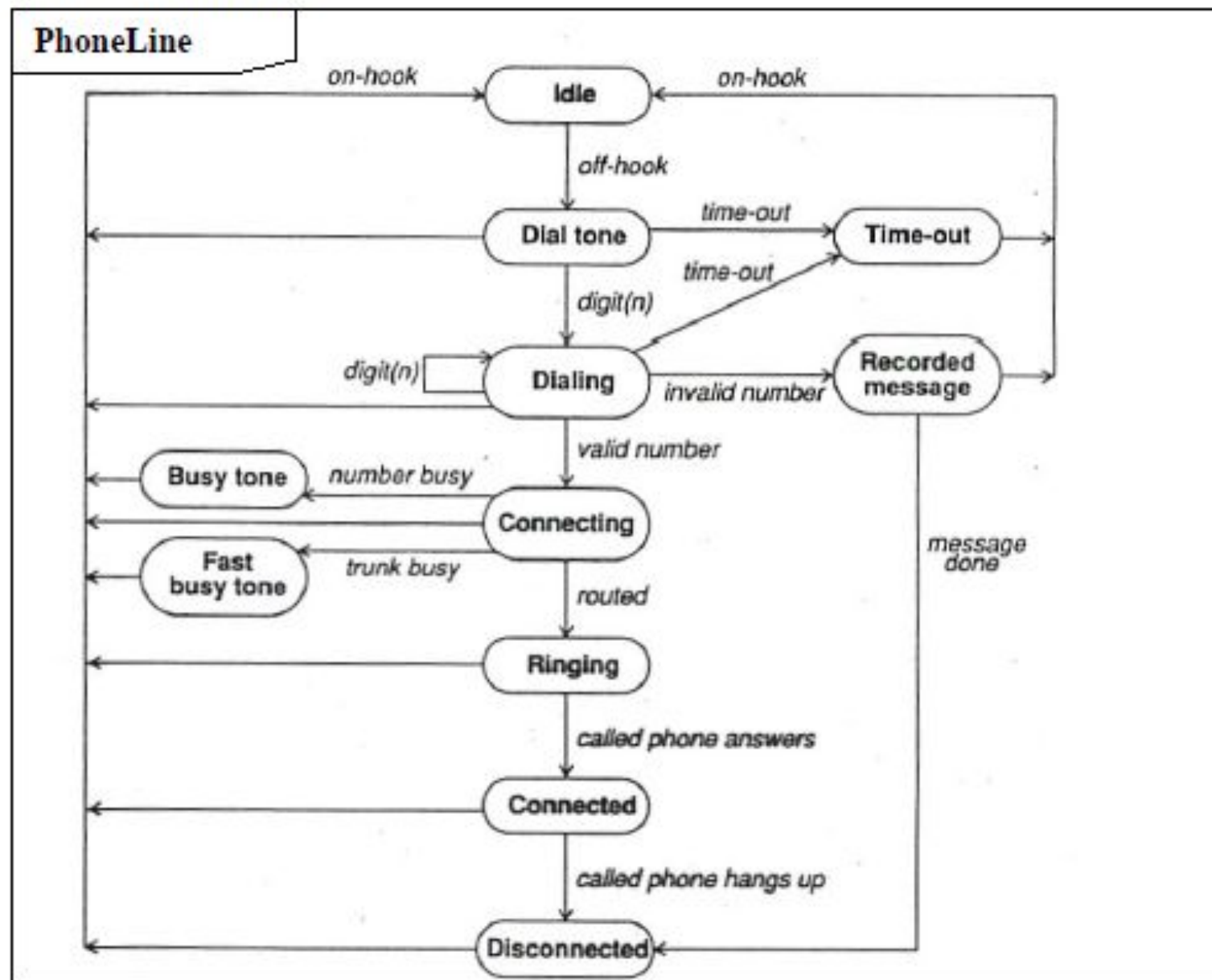
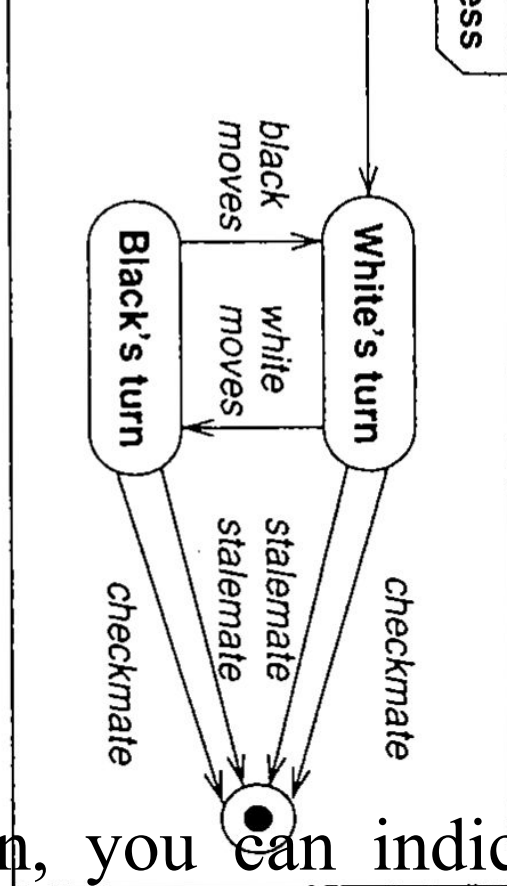


Figure 5.5 State diagram for phone line

❑ One shot state diagrams

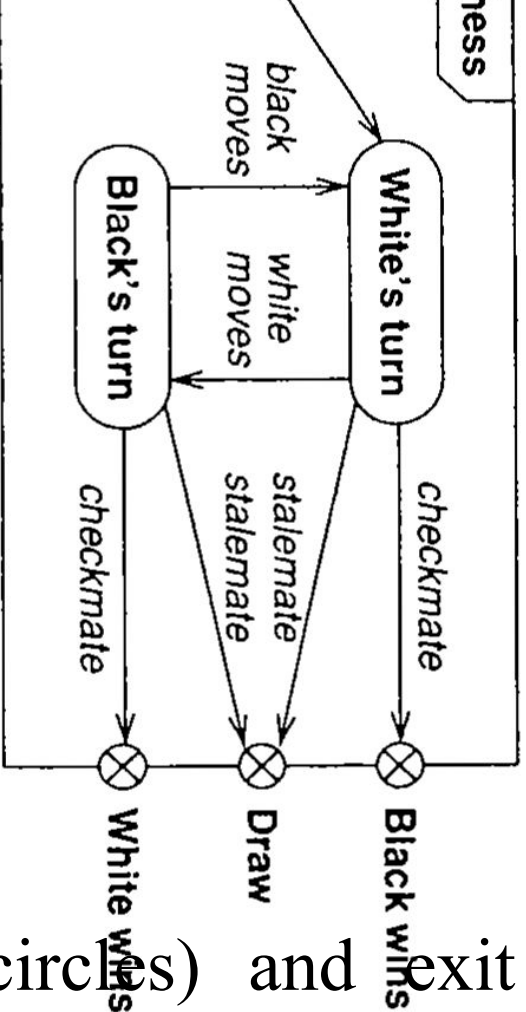
- ❑ State diagrams can represent continuous loops or one-shot life cycles
- ❑ Diagram for the phone line is a continuous loop
- ❑ One – shot state diagrams represent objects with finite lives and have initial and final states.
- ❑ The initial state is entered on creation of an object; entry of the final state implies destruction of the object.
- ❑ Fig below shows a simplified life cycle of a chess game with default initial state(solid circle) and a default final state(bull's eye).





state diagram for chess game. One-shot diagrams represent objects with finite lives.

- As an alternate notation, you can indicate initial and final states via entry and exit points.
- In fig below the *start* entry point leads to white's turn, and the chess game eventually ends with one of three possible outcomes.

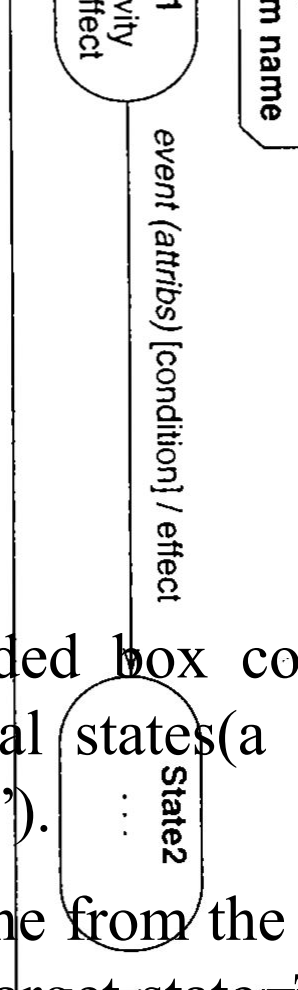


State diagram for chess game. You can also show one-shot diagrams by using entry and exit points.

- Entry points (hollow circles) and exit points (circles enclosing an "x") appear on the state diagram's perimeter and may be named.

□ Summary of Basic

5.11 Summary of basic notation for state diagrams.



tion

- State: Drawn as a rounded box containing an optional name. A special notation for initial states (a solid circle) and final states (a bull's eye or encircled "x").
- Transition: Drawn as a line from the origin state to the target site. An arrowhead points to the target state. The line may consist of several line segments.
- Event: A signal event is shown as a label on a transition and may be followed by parenthesized attributes. A change event is shown with the keyword *when* followed by a parenthesized Boolean expression

- A time event is shown with the keyword *when* followed by a parenthesized expression involving time or the keyword *after* followed by a parenthesized expression that evaluates to a time duration.
- State Diagram: Enclosed in a rectangular frame with the diagram name in a small pentagonal tag in the upper left corner.
- Guard condition: Optionally listed in square brackets after an event.
- Effects: Can be attached to a transition or state and are listed after a slash("/"). Multiple effects are separated with a comma and are performed concurrently.



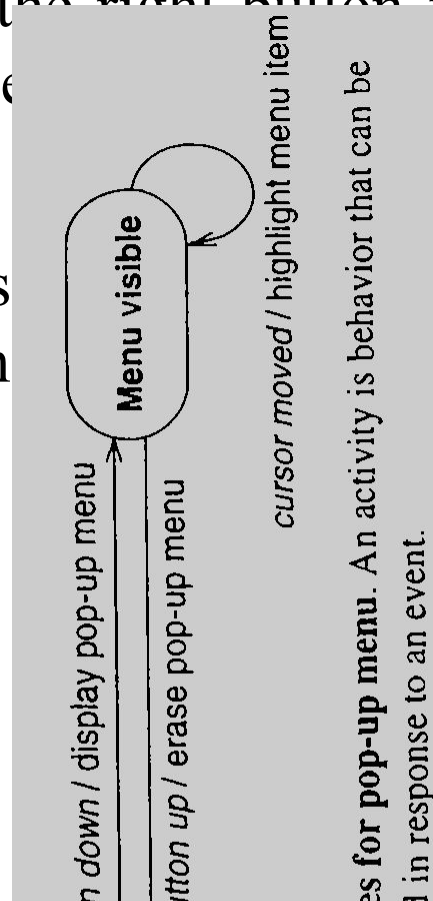
STATE DIAGRAM BEHAVIOR

□ Activity Effects

- An *effect* is a reference to a behavior that is executed in response to an event.
- An *activity* is the actual behavior that can be invoked by any number of effects.
- For Ex: *disconnect PhoneLine* might be an activity that is executed in response to an *OnHook* event.
- An activity may be performed upon a transition, upon the entry to or exit from a state, or upon some other event within a state.



- The notation for an activity is a slash (“/”) and the name of the activity, following the event that causes it.
- The keyword *do* is reserved for indicating an ongoing activity and may not be used as an event name.
- Fig shows the state diagram for a pop-up menu on a workstation. When the right button is depressed, the menu is displayed; when the button is released, the menu is erased.
- While the menu is visible, the menu is updated whenever the cursor moves. The highlighted menu item is

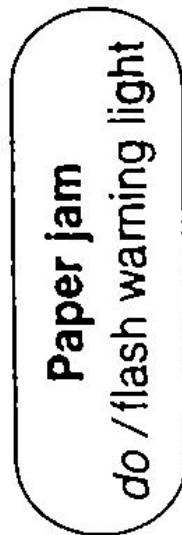


❑ Do- Activities

- A do-activity is an activity that continues for an extended time. By definition, a do-activity can only occur within a state and cannot be attached to a transition.
- For Ex: the warning light may flash during the *paper jam* state for a copy machine.
- Do-activities include continuous operations, such as displaying a picture on a television screen, as well as sequential operations that terminate by themselves after an interval of time, such as closing a valve.



- The notation “*do/*” denotes a do activity that may be performed for all or part of the duration that an object is in a state.
- A do-activity may be performed during its execution; a transition out of the state is possible.
- For Ex: a robot moving in a hallway encountering resistance, causing it to cease moving.



do-activity for a copy machine. A do-activity is an activity that continues for an extended time.

event that is received and may or may not cause a do-activity.

encounter resistance,



❑ Entry and Exit Activities

- ❑ As an alternative to showing activities on transitions, we can bind activities to entry or to exit from a state.
- ❑ Ex: Fig shows the control of a garage door opener. The user generates *depress* events with a pushbutton to open and close the door.
- ❑ Each event reverses the direction of the door, but for safety the door must open fully before it can be closed.
- ❑ The control generates *motor up* and *motor down* activities for the motor.
- ❑ The motor generates *door open* and *door closed* events when the motion has been completed. Both transitions entering the state *Opening* cause the door to open.



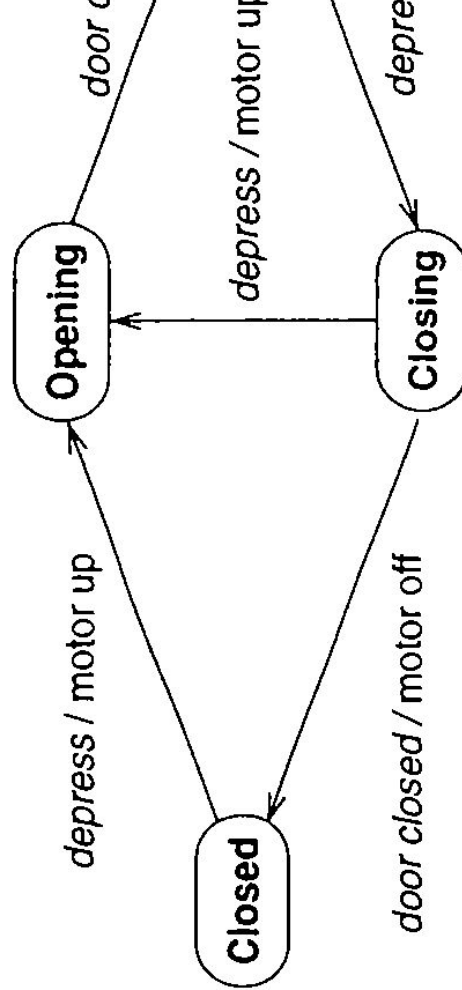


Figure 5.14 Activities on transitions. An activity may event that causes a transition.

- Fig below the state transition diagram for a door. An entry activity is performed every time the state is entered. The entry activity is the keyword *entry* and the activity is *door closed*.
- Whenever the state is entered, by any incoming transition, the entry activity is performed.
- If an incoming transition already has an activity, its activity is performed first.

- An exit activity keyword *exit* and
- Whenever the state is reached, the exit activity is performed.
- Any event can occur within a state and cause an activity to be performed.
- *Entry* and *exit* are only two examples of events that can occur

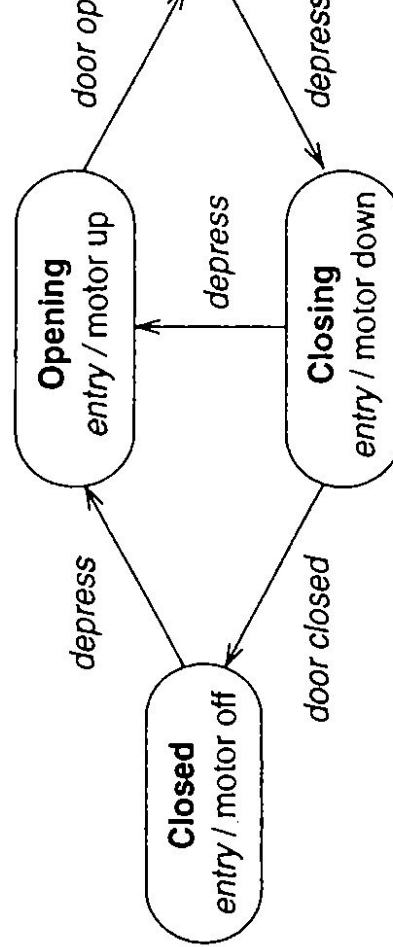


Figure 5.15 Activities on entry to states. An activity may occur within a state in response to an event that occurs within a state.

the box following the
going transition, the



within a state and a
it activities to be

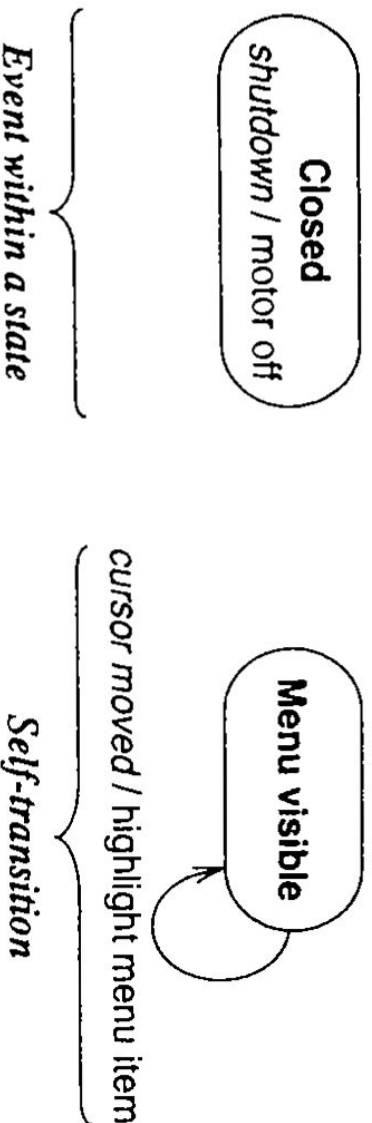


Figure 5.16 Event within a state vs. self-transition. A self-transition causes and exit activities to be executed. An event within a state does not

- There is a different self association can be executed.

❑ Completion Transition

- The purpose of a state is to perform a sequential activity. When the activity is completed, a transition to another state fires.
- An arrow without an event name indicates an automatic transition that fires when the activity associated with the source state is completed.
- Such unlabeled transitions are called *completed transitions* because they are triggered by the completion of activity in the source state.



□ Sending Signals

- An object can perform the activity of sending a signal to another object. A system of objects interacts by exchanging signals.
- The activity “send *target.S(attributes)*” sends signal S with the attributes to the target object or objects.
- For Ex: The phone line sends a *connect(phone number)* signal to the switcher when a complete phone number is dialed.
- A signal can be directed at a set of objects or a single object. If the target is a set of objects , each of them receives a separate copy of the signal concurrently.



- If the signal is always directed to the same object, the diagram can omit the target.
- If an object can receive signals from more than one object, the order in which concurrent signals are received may affect the final state, this is called race condition.
- For Ex: the door may or may not remain open if the button is pressed at about the time the door becomes fully open.
- A race condition is not necessarily a design error, but concurrent systems frequently contain unwanted race conditions that must be avoided by careful design.



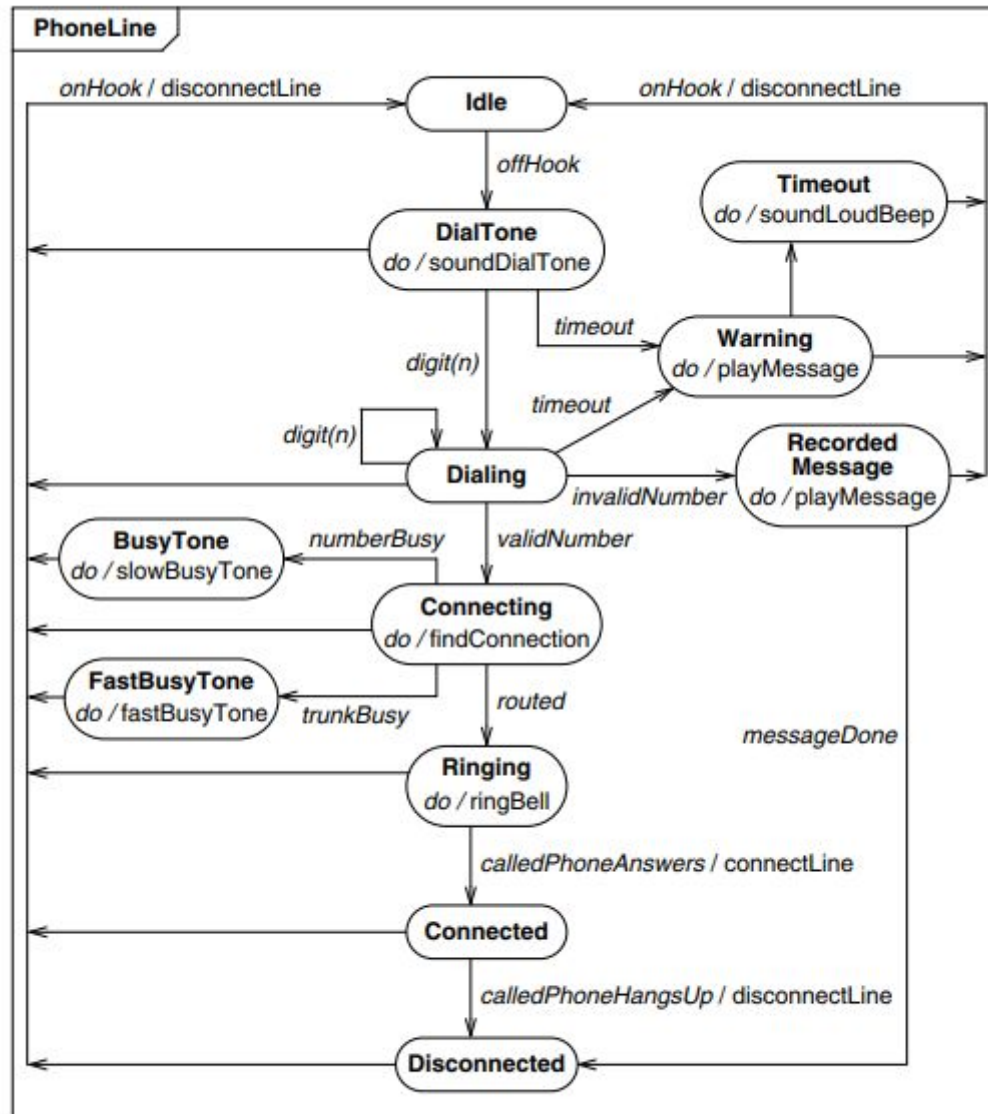


Figure 5.17 State diagram for phone line with activities. State diagrams let you express what objects do in response to events.